Side Activity 8: University Schema & Subqueries (with Join Practice)

CSCI3170 Introduction to Database Systems

Learning Objectives

In this Side Activity, you will learn:

- How to use the **university schema dataset** (DDL + data scripts).
- How to solve problems using **subqueries** with `IN`, `NOT IN`, and `EXISTS`.
- How to use **set comparison** (`ANY` / `ALL`), **scalar subqueries**, and **inline views** (subquery in `FROM`).
- How to recognize when a query is more readable as a **join** vs a **subquery**.

Part 0: Getting started (University schema dataset)

Please refer to the instructions in **Side Activity 3** to access your Oracle environment (SQL*Plus).

Step 0: Copy the Side Activity files to the server (same environment as Side Activity 3)

In Side Activity 3 (see "Part 1: Connecting to Oracle"), you SSH to `linux1.cse.cuhk.edu.hk` and then connect to SQL*Plus. If your `DDL.sql` and `smallRelationsInsertFile.sql` are on your **local computer**, you must copy them to the server first.

```
# Run on your LOCAL computer (macOS/Linux)
cd /path/to/your_folder
scp DDL.sql smallRelationsInsertFile.sql \
  {your_cse_id}@linux1.cse.cuhk.edu.hk:~/

# Run on the SERVER after SSH login (optional check)
ls -l DDL.sql smallRelationsInsertFile.sql
```

File location matters

In SQL*Plus, `@DDL.sql` runs a script from your **current working directory** on the server. If SQL*Plus says it cannot find the file, check where you copied it (`pwd`, `ls`) and run SQL*Plus from that folder.

Step 1: Create tables

Run the DDL script (tables: `student`, `instructor`, `course`, `section`, `takes`, `teaches`, etc.).

```
@DDL.sql
```

Step 2: Insert sample data

Load the small dataset.

```
@smallRelationsInsertFile.sql
```

Step 3: Quick sanity checks

```
SELECT COUNT(*) AS num_students   FROM student;
SELECT COUNT(*) AS num_courses    FROM course;
SELECT COUNT(*) AS num_sections   FROM section;
SELECT COUNT(*) AS num_takes_rows FROM takes;
```

Part 1: Quick join warm-up

Key Concept: Joins connect related rows across tables

In the university schema, `student` connects to `takes` by `student.ID = takes.ID`, and `takes` connects to `course/section` by matching `course_id` (and `sec_id, semester, year` for sections).

Demo 1: Student → Department (simple 2-table join)

This demo shows how to join `student` with `department` to display columns from both tables for each student (e.g., the department building).

```
SELECT s.ID, s.name, s.dept_name, d.building
FROM student s
JOIN department d ON d.dept_name = s.dept_name
ORDER BY s.dept_name, s.ID;
```

Demo 2: Student → Takes → Course (3 tables)

This demo shows a 3-table join where `takes` connects each `student` to the `course` they took.

```
SELECT s.ID, s.name, t.semester, t.year, c.course_id, c.title
FROM student s
JOIN takes  t ON t.ID = s.ID
JOIN course c ON c.course_id = t.course_id
ORDER BY s.ID, t.year, t.semester, c.course_id;
```

Demo 3: Add a filter (example: Spring 2018)

This demo adds a `WHERE` filter to restrict the join result to a specific term (Spring 2018).

```
SELECT s.ID, s.name, c.course_id, c.title, t.grade
FROM student s
JOIN takes  t ON t.ID = s.ID
JOIN course c ON c.course_id = t.course_id
WHERE t.semester = 'Spring' AND t.year = 2018
ORDER BY s.ID, c.course_id;
```

Part 2: Subqueries (basic patterns)

Key Concept: A subquery produces a value or a set

Common cases:

- **Scalar subquery**: returns exactly one value (used like a constant).
- **IN / NOT IN**: returns a set of values (membership test).

Demo 4: `IN` (students who took a specific course)

This demo uses an `IN` subquery to keep students whose `ID` appears in the set of `takes` rows for `CS-101`.

```
SELECT s.ID, s.name
FROM student s
WHERE s.ID IN (
  SELECT t.ID
  FROM takes t
  WHERE t.course_id = 'CS-101'
)
ORDER BY s.ID;
```

Demo 5: Same question using a join (compare readability)

This demo answers the same question as Demo 4 using a join; `DISTINCT` removes duplicates if multiple matching `takes` rows exist.

```
SELECT DISTINCT s.ID, s.name
FROM student s
JOIN takes t ON t.ID = s.ID
WHERE t.course_id = 'CS-101'
ORDER BY s.ID;
```

Demo 6: Scalar subquery (maximum instructor salary)

This demo uses a scalar subquery to compute one value (the maximum salary in the `instructor` table), then uses that value in a comparison.

```
SELECT i.ID, i.name, i.dept_name, i.salary
FROM instructor i
WHERE i.salary = (
  SELECT MAX(i2.salary)
  FROM instructor i2
```

```
)
ORDER BY i.ID;
```

Demo 7: One more `IN` / `NOT IN` example (courses with / without prerequisites)

This demo uses the `prereq` table. A course appears in `prereq.course_id` if it has at least one prerequisite.

```
-- Example A (IN): courses that HAVE a prerequisite record
SELECT c.course_id, c.title
FROM course c
WHERE c.course_id IN (
  SELECT p.course_id
  FROM prereq p
)
ORDER BY c.course_id;

-- Example B (NOT IN): courses that have NO prerequisite record
SELECT c.course_id, c.title
FROM course c
WHERE c.course_id NOT IN (
  SELECT p.course_id
  FROM prereq p
)
ORDER BY c.course_id;
```

Pitfall: `NOT IN` and NULL

If the subquery can return a NULL, `NOT IN (subquery)` may return **no rows** (because comparisons with NULL become UNKNOWN).
To avoid surprises, ensure the subquery column is **NOT NULL** (e.g., a primary key), or filter out NULLs inside the subquery.

```
... WHERE x NOT IN (
  SELECT y
  FROM T
  WHERE y IS NOT NULL
)
```

Demo 8: `ALL` / `ANY` (set comparison)

This demo illustrates set comparison: `ALL` means "greater than every value" in the subquery result, while `ANY` means "greater than at least one value".

Example A (ALL): instructors whose salary is higher than *every* instructor in the Comp. Sci. department.

```
SELECT i.ID, i.name, i.salary
FROM instructor i
WHERE i.salary > ALL (
  SELECT i2.salary
  FROM instructor i2
  WHERE i2.dept_name = 'Comp. Sci.'
)
```

```
ORDER BY i.salary DESC;
```

Example B (ANY): courses whose credits are higher than *at least one* course in the Comp. Sci. department.

```
SELECT c.course_id, c.title, c.credits
FROM course c
WHERE c.credits > ANY (
  SELECT c2.credits
  FROM course c2
  WHERE c2.dept_name = 'Comp. Sci.'
)
ORDER BY c.credits DESC, c.course_id;
```

Part 3: Subquery in `FROM` (inline view)

Key Concept: Inline view = use a subquery as a temporary table

Sometimes you want to compute an intermediate result (e.g., distinct students in 2018) and then aggregate or join it again. You can do this by putting a subquery in the `FROM` clause.

Demo 9: Count distinct students who took at least one course in 2018 (by department)

This demo first builds an inline view of distinct students who took at least one course in 2018, then groups by department to count those students.

```
SELECT x.dept_name, COUNT(*) AS num_students_2018
FROM (
  SELECT DISTINCT s.ID, s.dept_name
  FROM student s
  JOIN takes t ON t.ID = s.ID
  WHERE t.year = 2018
) x
GROUP BY x.dept_name
ORDER BY num_students_2018 DESC, x.dept_name;
```

Part 4: Try and study these queries (common subquery patterns)

These are **additional examples** for you to try and understand by yourself. They are **not for submission**.

Key Concept: Definition of `EXISTS` (and how it differs from `IN`)

`EXISTS (subquery)` is TRUE if the subquery returns **at least one row**; otherwise it is FALSE.
`IN` compares a value with a **set of values**. `EXISTS` checks whether a **matching row** exists.

1. **`IN` vs `EXISTS` (same question, two styles).** List instructors who have taught at least one section.

```
-- Style A: IN
SELECT i.ID, i.name, i.dept_name
FROM instructor i
WHERE i.ID IN (
  SELECT t.ID
  FROM teaches t
)
ORDER BY i.dept_name, i.ID;

-- Style B: EXISTS
SELECT i.ID, i.name, i.dept_name
FROM instructor i
WHERE EXISTS (
  SELECT 1
  FROM teaches t
  WHERE t.ID = i.ID
)
ORDER BY i.dept_name, i.ID;
```

2. `EXISTS` **(course offered at least once).** List all courses (`course_id, title`) that have been offered at least once (i.e., there exists a row in `section` with the same `course_id`).

```
SELECT c.course_id, c.title
FROM course c
WHERE EXISTS (
  SELECT 1
  FROM section s
  WHERE s.course_id = c.course_id
)
ORDER BY c.course_id;
```

3. **Max per group (highest salary per department).** For each department, output the instructor(s) who have the highest salary in that department. (If there is a tie, output all tied instructors.) Output `dept_name, ID, name, salary`.

```
SELECT i.dept_name, i.ID, i.name, i.salary
FROM instructor i
JOIN (
  SELECT dept_name, MAX(salary) AS max_salary
  FROM instructor
  GROUP BY dept_name
) m
  ON m.dept_name = i.dept_name
 AND m.max_salary = i.salary
ORDER BY i.dept_name, i.salary DESC, i.ID;
```

4. `NOT IN` **(anti-membership).** Find students who have never taken any course from their own department. Output `ID, name, dept_name`.

```
SELECT s.ID, s.name, s.dept_name
FROM student s
WHERE s.ID NOT IN (
  SELECT DISTINCT t.ID
  FROM takes t
```

```
  JOIN course  c  ON c.course_id = t.course_id
  JOIN student s2 ON s2.ID = t.ID
  WHERE c.dept_name = s2.dept_name
)
ORDER BY s.dept_name, s.ID;
```

Part 5: Self Challenge Exercises (not graded)

Submit your SQL answers (8 queries) on the **University schema** for the following questions.

1. Find the names of all the instructors from Biology department.
2. Find the names of courses in Computer science department which have 3 credits.
3. For the student with ID `12345` (or any other value), show all `course_id` and `title` of all courses registered for by the student.
4. As above, but show the total number of credits for such courses (taken by that student). Don't display the `tot_cred` value from the `student` table; use SQL aggregation on courses taken by the student.
5. As above, but display the total credits for each of the students, along with the ID of the student; don't bother about the name of the student. (Don't bother about students who have not registered for any course; they can be omitted.)
6. Find the names of all students who have taken any Comp. Sci. course ever (there should be no duplicate names).
7. Display the IDs of all instructors who have never taught a course. Notes: (1) Oracle uses the keyword `MINUS` in place of `EXCEPT`; (2) interpret "taught" as "taught or is scheduled to teach".
8. As above, but display the names of the instructors also, not just the IDs.

Reminder (Oracle set difference)

If you want to solve Q7–Q8 using set difference, Oracle uses `MINUS` instead of `EXCEPT`.

End of Side Activity 8.